



BICOL UNIVERSITY
COLLEGE OF SCIENCE
Legazpi City, Albay
A.Y. 2025 - 2026



Course: IT 106 - Application Development and Emerging Technologies

Activity: Laboratory 4: REST API

Group Members:

- Mheea Rose J. Andes
- Frederick B. Baldano
- Edgar Neil M. Bayoca
- Ryan M. Lumbis
- Jose Manuel G. Labalan

System: Academic Grade Tracker

Clients:

- Python console app
- Web browser app

Source Code Link: <https://github.com/ltsRyan504/REST-API.git>

API OVERVIEW

The GradeTrack API is a REST-style PHP API for user authentication, student management, subject grades, and GWA calculation. Public endpoints include `POST /register`, `POST /login`, and `POST /logout`. Protected endpoints allow users to manage profiles, students, subjects, and compute GWA, such as `GET /me`, `GET /students`, `POST /students`, `GET /students/{id}/gwa`, and subject CRUD routes. The API uses parameters like `full_name`, `username`, `password`, `name`, `year_level`, `subject_name`, and `grade`. It handles validation, Bearer-token authentication, duplicate checks, grade verification, automatic GWA computation, and academic remarks like Passed, Cum Laude, or Failed.

```
define('TOKEN_EXPIRY', 86400);  
const VALID_GRADES = [1.0, 1.25, 1.5, 1.75, 2.0, 2.25, 2.5, 2.75, 3.0,  
5.0];  
  
$resource = $parts[0] ?? null;  
$id = isset($parts[1]) ? (int) $parts[1] : null;  
$sub = $parts[2] ?? null;  
$subParam = isset($parts[3]) ? urldecode($parts[3]) : null;
```

The API also applies business logic beyond simple CRUD operations. During login, it generates a session token and requires that token for protected routes. For student records, it can create, retrieve, update, and delete data. For subjects, it validates that only accepted grade values are stored and prevents duplicate subject names for the same student. It then computes the General Weighted Average by averaging all subject grades and returns a matching academic remark.

```
function calculateGWA(array $subjects): ?float
{
    if (empty($subjects)) return null;
    $total = array_sum(array_column($subjects, 'grade'));
    return round($total / count($subjects), 4);
}

function getRemarks(?float $gwa): string
{
    if ($gwa === null) return 'No grades recorded';
    if ($gwa <= 1.20) return 'Summa Cum Laude';
    if ($gwa <= 1.45) return 'Magna Cum Laude';
    if ($gwa <= 1.75) return 'Cum Laude';
    if ($gwa <= 3.00) return 'Passed';
    return 'Failed';
}

if ($method === 'POST' && $resource === 'login') {}
if ($method === 'GET' && $resource === 'me') {}
if ($method === 'GET' && $resource === 'students') {}
if ($method === 'POST' && $resource === 'students') {}
if ($method === 'GET' && $sub === 'gwa') {}
if ($method === 'POST' && $sub === 'subjects')
```

CLIENT APPLICATION

The client application is a browser-based frontend built with HTML, CSS, and JavaScript. It serves as the user interface that communicates with the backend API using the Fetch API. Through this client, users can register, log in, view students, search records, add new students, edit student information, manage subjects and grades, and view the computed GWA and remarks. The frontend focuses on displaying data and handling user interaction, while the backend handles validation, authentication, and computation.

```
const API_BASE = (() => {
    // Works regardless of subfolder name in htdocs
```

```
const path = window.location.pathname.replace(/\/[^\/]*$/, '');
return `${window.location.origin}${path}/api.php`;
})();
```

The client stores the login token in `localStorage` so that protected requests can include the `Authorization: Bearer` header automatically. It sends HTTP requests to the API, receives JSON responses, and updates the interface dynamically based on the returned data.

```
1  async function api(method, path, body = null) {
2  const opts = {
3    method,
4    headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
5  };
6
7  const token = localStorage.getItem('gs_token');
8  if (token) opts.headers['Authorization'] = `Bearer ${token}`;
9  if (body) opts.body = JSON.stringify(body);
10
11  const res = await fetch(`${API_BASE}${path}`, opts);
12  return await res.json();
13  }
14
```

The frontend loads student data from the API and renders it on the page. When a user selects a student, the application displays the student's subjects, grades, GWA, and remarks. It also allows users to add and update records by sending POST, PUT, and DELETE requests, which shows how one client application can fully consume the REST API.

```
1  async function LoadStudents() {
2    const res = await api('GET', '/students');
3    students = res.data;
4    renderStudentsList();
5  }
6
7  async function LoadStudentDetail(id) {
8    const res = await api('GET', `/students/${id}`);
9    renderStudentDetail(res.data);
10  }
11
```

The Python file is a second client application that consumes the REST API. Unlike the web client built with HTML, CSS, and JavaScript, the Python client is a command-line program that sends HTTP requests to the PHP backend, receives JSON responses, and displays the

returned data in the terminal. This shows that the same REST API can be used by different client applications, not just a browser interface. In this project, the Python client can register and log in users, list students, add students, add subject grades, and view a student's GWA.

```
class SchoolAPIClient:
    def __init__(self, base_url):
        self.base_url = base_url.rstrip("/")
        self.token = None
        self.user = None

    def _request(self, method, path, payload=None, use_auth=False):
        url = f"{self.base_url}{path}"
        headers = {"Accept": "application/json"}

        if use_auth:
            headers["Authorization"] = f"Bearer {self.token}"

        req = request.Request(url, headers=headers, method=method)
        with request.urlopen(req) as response:
            return json.loads(response.read().decode("utf-8"))
```

The Python client follows the same API flow as the web client. It connects to the API base URL, sends requests to endpoints like /login, /students, and /students/{id}/gwa, and uses the Bearer token for protected routes. This makes it a clear example of a non-browser client that still works with the same backend.

```
class SchoolAPIClient:
    def __init__(self, base_url):
        self.base_url = base_url.rstrip("/")
        self.token = None
        self.user = None

    def _request(self, method, path, payload=None, use_auth=False):
        url = f"{self.base_url}{path}"
        headers = {"Accept": "application/json"}

        if use_auth:
            headers["Authorization"] = f"Bearer {self.token}"

        req = request.Request(url, headers=headers, method=method)
        with request.urlopen(req) as response:
            return json.loads(response.read().decode("utf-8"))
```

PROJECT DEMO : Python Console App

1. Authorization/Login Page

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php
Logged in as: Guest

Choose an action:
[1] Register
[2] Login
[0] Exit
Enter choice: █
```

a. Register

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php
Full name: Juan Miguel Lopez
Username: JML
Password:

Account created successfully! You can now log in.

Press Enter to proceed...█
```

b. Login

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php
Username: JML
Password:

Login successful!
Welcome, Juan Miguel Lopez!

Press Enter to proceed...█
```

c. Exit

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php

Exiting Python client.
PS C:\xampp\htdocs\REST-API> █
```

2. Main Menu

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php
Logged in as: Juan Miguel Lopez

Choose an action:
[1] Show My Account
[2] List Students
[3] Add Student
[4] View Student GWA
[5] Logout
[0] Exit
Enter choice: █
```

a. Show my Account

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php

ID: 5
Name: Juan Miguel Lopez
Username: JML
Role: teacher

Press Enter to proceed... █
```

b. List Students

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php

ID: 1 | Name: Juan Dela Ceuz | Year: 2nd Year | GWA: 3.00 | Remarks: Passed
ID: 2 | Name: Frederick Baldano | Year: 4 | GWA: N/A | Remarks: No grades recorded
ID: 3 | Name: Frederick | Year: 4 | GWA: N/A | Remarks: No grades recorded
ID: 4 | Name: sajdnasjd | Year: 3 | GWA: N/A | Remarks: No grades recorded
ID: 5 | Name: JM Labalan | Year: 3 | GWA: N/A | Remarks: No grades recorded

Press Enter to proceed...|
```

c. Add Student

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php
Student name: Marvin Marfil
Year level: 3

Student "Marvin Marfil" created successfully.

Press Enter to proceed...|
```

d. Add Subject Grade

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php
Student ID: 1
Subject name: ADET
Grade (e.g. 1.75): 1.0

Subject "ADET" added.

Press Enter to proceed...|
```

e. View Student GWA

```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php
Student ID: 1

Student: Juan Dela Ceuz
Year Level: 2nd Year
GWA: 2.0000
Remarks: Passed
Total Subjects: 2

Press Enter to proceed...|
```

f. Logout

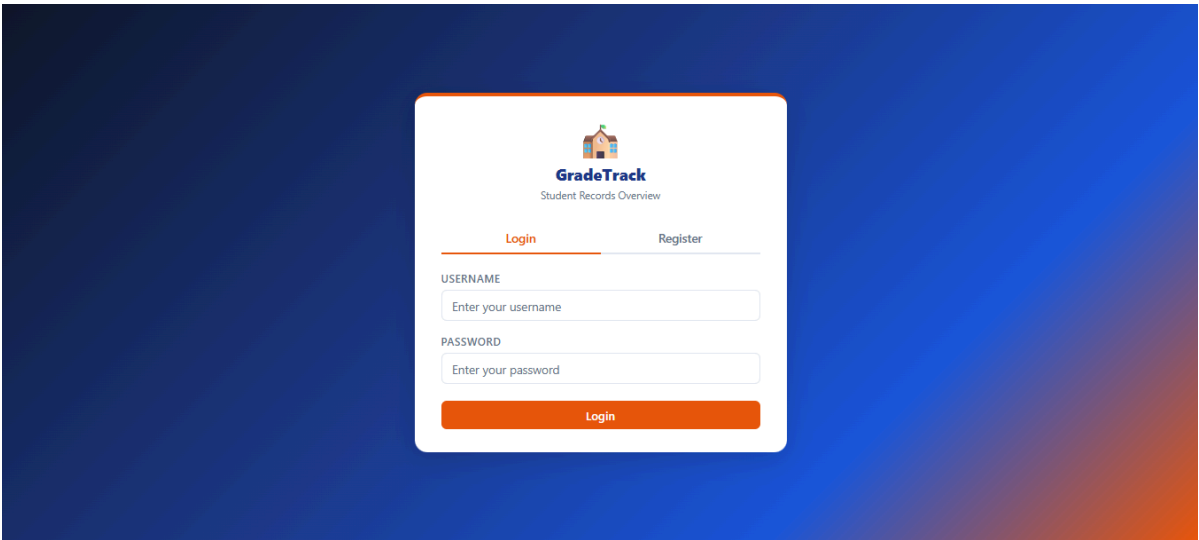
```
=====
GradeTrack - Python Client
=====
API Base URL: http://localhost/REST-API/api.php

Logged out successfully.

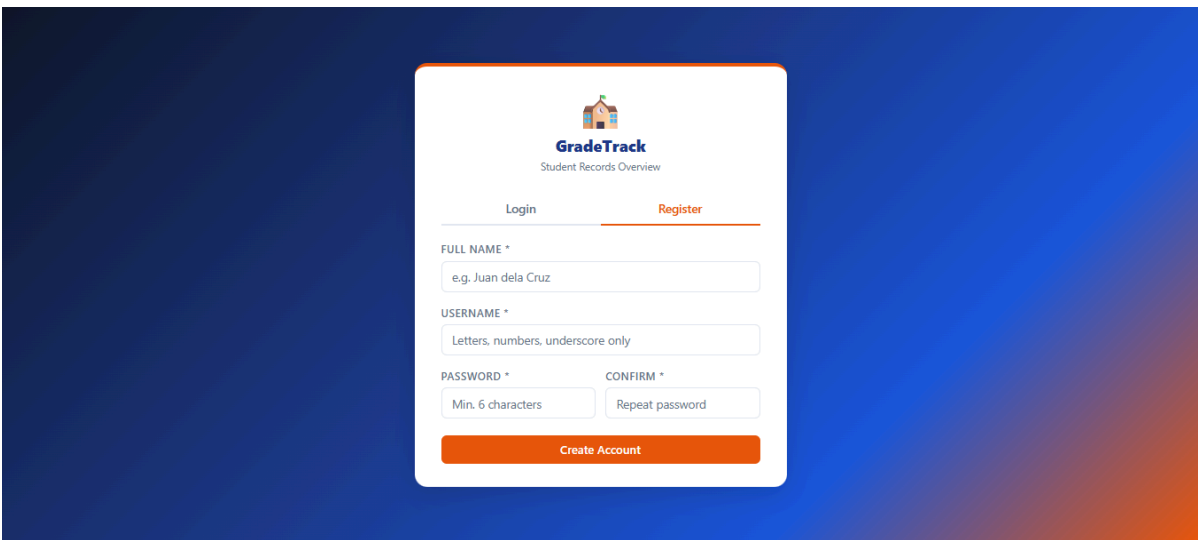
Press Enter to proceed...|
```


PROJECT DEMO : Web Browser App

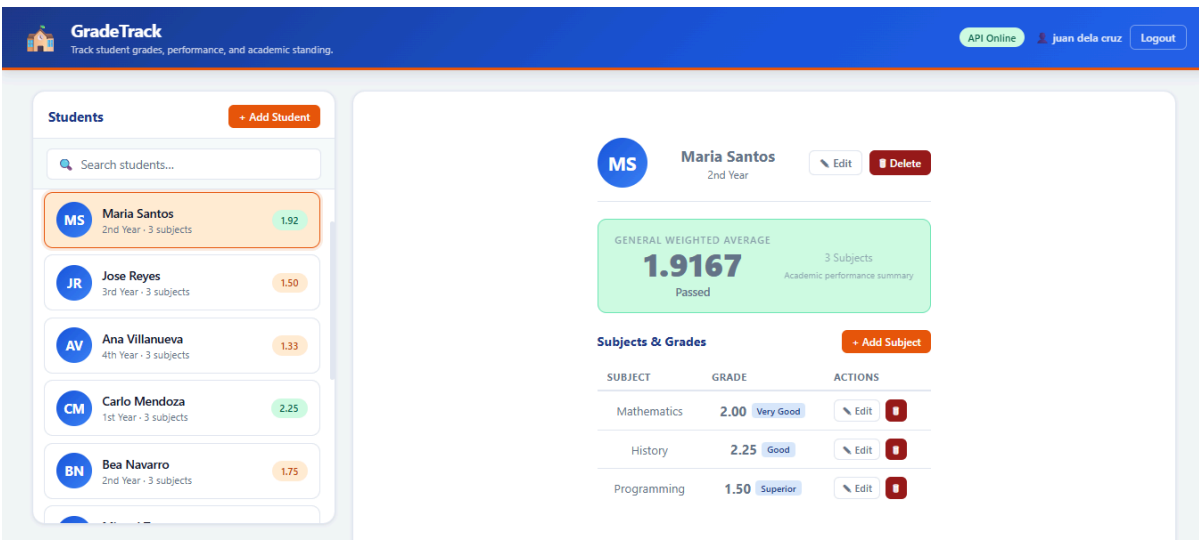
1. Login Page



2. Register Page



3. Home Menu



4. Add Student

Add New Student

FULL NAME *

e.g. Juan dela Cruz

YEAR LEVEL

— Select Year Level —

Cancel

Add Student

5. View Student

GradeTrack

Track student grades, performance, and academic standing.

API Online

Logout

Students

+ Add Student

Search students...

JD

Juan Dela Cruz

2nd Year - 1 subject

3.00

FB

Frederick Baldano

4 - 0 subjects

—

F

Frederick

4 - 0 subjects

—

S

sajdnasjd

3 - 0 subjects

—

JL

JM Labalan

3 - 0 subjects

—

A

asda

4 - 0 subjects

—

MM

Marvin Marfil

3 - 0 subjects

—

FB

Frederick Baldano

4

Edit

Delete

GENERAL WEIGHTED AVERAGE

0 Subjects

No grades recorded

Academic performance summary

Subjects & Grades

+ Add Subject

SUBJECT	GRADE	ACTIONS
No subjects added yet. Click + Add Subject.		

6. Edit Student

Edit Student

FULL NAME *

Frederick Baldano

YEAR LEVEL

— Select Year Level —

Cancel

Save Changes

7. Add Subject/GWA

Add Subject

SUBJECT NAME *

Data Structure and Algorithms

GRADE *

— Select Grade —

Cancel

Add Subject

FB

Frederick Baldano

4

Edit

Delete

GENERAL WEIGHTED AVERAGE

3.0000

Passed

1 Subject

Academic performance summary

Subjects & Grades

+ Add Subject

SUBJECT	GRADE	ACTIONS
Data Structure and Algorithms	3.00 Passing	Edit

8. Remove Subject

Remove Subject

Remove **Data Structure and Algorithms** from this student's record?

Cancel

Yes, Remove